cnn_linearity_compression

January 15, 2026

1 Detecting inherent linearity in large CNNs

This Notebook serves to experiment with detecting inherent linearity in ResNets. These experiments will be run on smaller instances of ResNets to test functionality and provide evidence for a feasibility study as part of the graduation preparation phase. Larger experiments for the final paper(s) will be handled using Python files in this repository.

[1]:
```
import torch
from torchvision import transforms
from torchvision.models import resnet18, ResNet18_Weights
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader
from torch.utils.data import Subset
import numpy as np
from tqdm import tqdm

# --- CUDA sanity ---
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
print(torch.version.cuda)

torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True

print(torch.backends.cuda.matmul.allow_tf32)
print(torch.backends.cudnn.allow_tf32)
```

```
True
NVIDIA GeForce RTX 4080 SUPER
13.0
True
True
```

[2]:
```
def make_subset(dataset, fraction=0.1, seed=42):
    np.random.seed(seed)
    indices = np.random.choice(
        len(dataset),
        int(len(dataset) * fraction),
```

```
        replace=False
    )
    return Subset(dataset, indices)


train_tfms = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    ),
])

val_tfms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    ),
])

train_ds = ImageFolder("./data/imagenet_train", transform=train_tfms)
train_ds = make_subset(train_ds, fraction=0.05)
val_ds = ImageFolder("./data/imagenet_val", transform=val_tfms)

train_loader = DataLoader(
    train_ds, batch_size=256, shuffle=True,
    num_workers=0, pin_memory=True
)

val_loader = DataLoader(
    val_ds, batch_size=128, shuffle=False,
    num_workers=0, pin_memory=True
)

model = resnet18(weights=ResNet18_Weights.DEFAULT).cuda().eval()
```

1.0.1 Verifying the model and data loader work correctly

[3]:
```
def verify_model_and_data_loader(model, data_loader, device="cuda",␣
 ↪num_batches=1):
    """
    Verify that the model and data loader work correctly by performing a forward
```

```
    pass on ImageNet-sized inputs.

    Args:
        model: ImageNet-native neural network (e.g. ResNet-18)
        data_loader: DataLoader over ImageNet val (ImageFolder)
        device: Device to run computations on
        num_batches: Number of batches to run

    Returns:
        torch.Tensor: Concatenated outputs from the first num_batches
    """

    # --- move model once ---
    model = model.to(device).eval()

    outputs = []

    with torch.no_grad():
        for inputs, _ in tqdm(data_loader, desc="Doing ImageNet forward passes",␣
↪leave=False):
            # ImageNet inputs: N×3×224×224
            inputs = inputs.to(device)

            output = model(inputs)
            outputs.append(output)

            if len(outputs) >= num_batches:
                break

    return torch.cat(outputs, dim=0)

outputs = verify_model_and_data_loader(model, val_loader)
print(outputs.shape)
```

```
torch.Size([128, 1000])
```

1.1 Computing the mean of preactivations per ReLU layer

"**Mean of preactivations** $\bar{p}^l$ We denote the distribution of inputs z to a nonlinear activation function $f(z)$ as the preactivations. For each activation function/node, we compute the mean of the preactivations, and then we compute another mean of these values per layer l: $\bar{p}^l = \frac{1}{M}\sum_{i=1}^{M}\left(\frac{1}{N}\sum_{s=1}^{N} z_{s,i}^l\right)$, with M the number of nodes in layer l and N the number of samples, and $z_{s,i}^l$ the preactivation value for sample s at node i at layer l. We compute this value through randomly selecting 500 samples of the input data instead of the whole dataset, which significantly

reduces the computational cost." (Pinson et al., 2024, p. 3).

In case there is BatchNormalization applied between the convolution and the activation function, the mean of the preactivations will be approximately 0, due to the normalization. However, BN has two learned parameters per channel, namely a scaling and a shifting parameter. The shifting parameter can be used to recover the mean of the preactivations before BN. Therefore, in case of BN, we will use the shifting parameter as the mean of the preactivations. (Pinson et al., 2024)

[4]:
```
def retrieve_mean_preactivations(model, data_loader, device='cuda'):
    """Compute the mean of preactivations for each ReLU layer in the model.
    Function does this by reading the learned shift parameter of the preceding␣
↪BatchNormalization layer,
    as the shift will cause an inaccurate mean when computed at the ReLU layer.
    Returns a dictionary with layer names as keys and mean preactivation values␣
↪as values.
    Args:
        model: The neural network model.
        data_loader: DataLoader for the input data.
        device: Device to run the computations on.

    Returns:
        dict: A dictionary with layer names as keys and mean preactivation␣
↪values as values.
    """
    model.to(device)
    model.eval()

    batchnorms = {}
    mean_preactivations = {}
    hooks = []

    def get_hook(name):
        def hook(module, input, output):
            # If the layer is a proper BatchNorm and it isn't the last in the␣
↪block, save it with its shift parameter
            if isinstance(module, torch.nn.BatchNorm2d) and 'downsample' not in␣
↪name and name.split('bn')[-1] < '2':
                shift = module.bias.mean().item()
                layer = name.split('bn')[0]  # Get the layer name before .bn
                batchnorms[layer] = shift
            # If the layer is ReLU, check if there was a preceding BatchNorm
            elif isinstance(module, torch.nn.ReLU):
                layer = name.split('relu')[0]  # Get the layer name before .relu
                if layer in batchnorms:
                    mean_preactivations[name] = batchnorms[layer]
                else:
                    # Compute mean preactivation directly
```

```
                    print(f"Computing mean preactivation for layer {name}␣
↪without preceding BatchNorm.")
                    preactivations = input[0]
                    mean_preactivation = preactivations.mean().item()
                    mean_preactivations[name] = mean_preactivation
        return hook

    for name, module in tqdm(model.named_modules(), desc="Registering hooks",␣
↪leave=False):
        if isinstance(module, (torch.nn.ReLU, torch.nn.BatchNorm2d)):
            hooks.append(module.register_forward_hook(get_hook(name)))
    with torch.no_grad():
        for inputs, _ in tqdm(data_loader, desc="Computing mean preactivations",␣
↪leave=False):
            inputs = inputs.to(device)
            model(inputs)
            break  # Only need one batch for mean preactivations
    for hook in tqdm(hooks, desc="Removing hooks", leave=False):
        hook.remove()
    return mean_preactivations


mean_preacts = retrieve_mean_preactivations(model, val_loader)
print(mean_preacts)
```

```
{'relu': 0.18112018704414368, 'layer1.0.relu': -0.0341365709900856,
'layer1.1.relu': -0.08357393741607666, 'layer2.0.relu': -0.06734631955623627,
'layer2.1.relu': -0.2102503776550293, 'layer3.0.relu': -0.11478555202484131,
'layer3.1.relu': -0.23746797442436218, 'layer4.0.relu': -0.22572244703769684,
'layer4.1.relu': -0.24174048006534576}
```

The mean of preactivations can indicate whether the preceding layer is operating in a linear regime. For ReLU activations, a mean preactivation close to zero indicates that the layer is likely operating in a more linear regime, as many inputs will be negative and thus output zero. Conversely, a mean preactivation significantly greater than zero suggests that the layer is operating in a more nonlinear regime, as more inputs will be positive and thus pass through the ReLU unchanged. Below we copy the mean preactivations to the preceding Conv2d layer for easier analysis later on.

[5]:
```
mean_preacts_conv = {}
for name, module in model.named_modules():
    if isinstance(module, torch.nn.ReLU):
        mean = mean_preacts.get(name, 0.0)
        layer = name.split('relu')[0]  # Get the layer name before .relu
        mean_preacts_conv[layer + 'conv1'] = mean  # Assign to preceding Conv2d␣
↪layer
```

[6]:
```python
# Print all model layers neatly, and remark the mean preactivations for ReLU and␣
↪the two learned parameters of BatchNorm2d
for name, module in model.named_modules():
    if isinstance(module, torch.nn.ReLU):
        mean = mean_preacts.get(name, 0.0)
        print(f"Layer: {name} (ReLU), Mean preactivation: {mean:.6f}")
    elif isinstance(module, torch.nn.BatchNorm2d):
        gamma = model.state_dict()[f"{name}.weight"].mean().item()
        beta = model.state_dict()[f"{name}.bias"].mean().item()
        print(f"Layer: {name} (BatchNorm2d), gamma (scale): {gamma:.6f}, beta␣
↪(shift/mean preactivation): {beta:.6f}")
    elif isinstance(module, torch.nn.Conv2d):
        mean = mean_preacts_conv.get(name, None)
        if mean is not None:
            print(f"Layer: {name} (Conv2d), Mean preactivation (from following␣
↪ReLU): {mean:.6f}")
    else:
        print(f"Layer: {name} ({module.__class__.__name__})")
# This can be used to verify that the mean preactivations for the ReLU layers␣
↪were properly copied from preceding BatchNorm layers.
```

```
Layer:  (ResNet)
Layer: conv1 (Conv2d), Mean preactivation (from following ReLU): 0.181120
Layer: bn1 (BatchNorm2d), gamma (scale): 0.257577, beta (shift/mean
preactivation): 0.181120
Layer: relu (ReLU), Mean preactivation: 0.181120
Layer: maxpool (MaxPool2d)
Layer: layer1 (Sequential)
Layer: layer1.0 (BasicBlock)
Layer: layer1.0.conv1 (Conv2d), Mean preactivation (from following ReLU):
-0.034137
Layer: layer1.0.bn1 (BatchNorm2d), gamma (scale): 0.339601, beta (shift/mean
preactivation): -0.034137
Layer: layer1.0.relu (ReLU), Mean preactivation: -0.034137
Layer: layer1.0.bn2 (BatchNorm2d), gamma (scale): 0.333055, beta (shift/mean
preactivation): 0.003463
Layer: layer1.1 (BasicBlock)
Layer: layer1.1.conv1 (Conv2d), Mean preactivation (from following ReLU):
-0.083574
Layer: layer1.1.bn1 (BatchNorm2d), gamma (scale): 0.328692, beta (shift/mean
preactivation): -0.083574
Layer: layer1.1.relu (ReLU), Mean preactivation: -0.083574
Layer: layer1.1.bn2 (BatchNorm2d), gamma (scale): 0.392430, beta (shift/mean
preactivation): -0.029984
Layer: layer2 (Sequential)
Layer: layer2.0 (BasicBlock)
Layer: layer2.0.conv1 (Conv2d), Mean preactivation (from following ReLU):
```

```
-0.067346
Layer: layer2.0.bn1 (BatchNorm2d), gamma (scale): 0.316419, beta (shift/mean
preactivation): -0.067346
Layer: layer2.0.relu (ReLU), Mean preactivation: -0.067346
Layer: layer2.0.bn2 (BatchNorm2d), gamma (scale): 0.327573, beta (shift/mean
preactivation): -0.003555
Layer: layer2.0.downsample (Sequential)
Layer: layer2.0.downsample.1 (BatchNorm2d), gamma (scale): 0.195085, beta
(shift/mean preactivation): -0.003555
Layer: layer2.1 (BasicBlock)
Layer: layer2.1.conv1 (Conv2d), Mean preactivation (from following ReLU):
-0.210250
Layer: layer2.1.bn1 (BatchNorm2d), gamma (scale): 0.321264, beta (shift/mean
preactivation): -0.210250
Layer: layer2.1.relu (ReLU), Mean preactivation: -0.210250
Layer: layer2.1.bn2 (BatchNorm2d), gamma (scale): 0.282911, beta (shift/mean
preactivation): -0.151284
Layer: layer3 (Sequential)
Layer: layer3.0 (BasicBlock)
Layer: layer3.0.conv1 (Conv2d), Mean preactivation (from following ReLU):
-0.114786
Layer: layer3.0.bn1 (BatchNorm2d), gamma (scale): 0.312275, beta (shift/mean
preactivation): -0.114786
Layer: layer3.0.relu (ReLU), Mean preactivation: -0.114786
Layer: layer3.0.bn2 (BatchNorm2d), gamma (scale): 0.320250, beta (shift/mean
preactivation): -0.030766
Layer: layer3.0.downsample (Sequential)
Layer: layer3.0.downsample.1 (BatchNorm2d), gamma (scale): 0.082104, beta
(shift/mean preactivation): -0.030766
Layer: layer3.1 (BasicBlock)
Layer: layer3.1.conv1 (Conv2d), Mean preactivation (from following ReLU):
-0.237468
Layer: layer3.1.bn1 (BatchNorm2d), gamma (scale): 0.278211, beta (shift/mean
preactivation): -0.237468
Layer: layer3.1.relu (ReLU), Mean preactivation: -0.237468
Layer: layer3.1.bn2 (BatchNorm2d), gamma (scale): 0.245850, beta (shift/mean
preactivation): -0.163723
Layer: layer4 (Sequential)
Layer: layer4.0 (BasicBlock)
Layer: layer4.0.conv1 (Conv2d), Mean preactivation (from following ReLU):
-0.225722
Layer: layer4.0.bn1 (BatchNorm2d), gamma (scale): 0.264318, beta (shift/mean
preactivation): -0.225722
Layer: layer4.0.relu (ReLU), Mean preactivation: -0.225722
Layer: layer4.0.bn2 (BatchNorm2d), gamma (scale): 0.424319, beta (shift/mean
preactivation): -0.197634
Layer: layer4.0.downsample (Sequential)
Layer: layer4.0.downsample.1 (BatchNorm2d), gamma (scale): 0.250642, beta
```

```
(shift/mean preactivation): -0.197634
Layer: layer4.1 (BasicBlock)
Layer: layer4.1.conv1 (Conv2d), Mean preactivation (from following ReLU):
-0.241740
Layer: layer4.1.bn1 (BatchNorm2d), gamma (scale): 0.288600, beta (shift/mean
preactivation): -0.241740
Layer: layer4.1.relu (ReLU), Mean preactivation: -0.241740
Layer: layer4.1.bn2 (BatchNorm2d), gamma (scale): 1.853810, beta (shift/mean
preactivation): 0.273822
Layer: avgpool (AdaptiveAvgPool2d)
Layer: fc (Linear)
```

1.2 Pruning the model and identifying the pruned layers

To attempt pruning of the model, we can merge convolutional layers with a (near-)linear ReLU between them. We do this by removing the ReLU and applying the Layer Folding technique from Dror et al. (2021). The technique is described in the paper as follows:

For $g_1(\curvearrowleft) = \{\sum_{i=1}^{c_\mathbb{X}} \mathbb{W}_1^{i,j} * \curvearrowleft_i\}_{j=1}^{c_\mathbb{Y}}$ and $g_2(\curvearrowright) = \{\sum_{j=1}^{c_\mathbb{Y}} \mathbb{W}_2^{j,m} * \curvearrowright_j\}_{m=1}^{c_\mathbb{Z}}$, where $\curvearrowleft \in \mathbb{R}^{h\times w\times c_\mathbb{X}}$, $\curvearrowright \in \mathbb{R}^{h\times w\times c_\mathbb{Y}}$, and $\digamma \in \mathbb{R}^{h\times w\times c_\mathbb{Z}}$ are the input, intermediate, and output feature maps, respectively, and $\mathbb{W}_1 \in \mathbb{R}^{k_1\times k_1\times c_\mathbb{Y}\times c_\mathbb{X}}$ and $\mathbb{W}_2 \in \mathbb{R}^{k_2\times k_2\times c_\mathbb{Z}\times c_\mathbb{Y}}$ are the convolutional kernels of g_1 and g_2, respectively, we can define a new convolutional layer $g_{fold}(\curvearrowleft) = \{\sum_{i=1}^{c_\mathbb{X}} \left(\sum_{j=1}^{c_\mathbb{Y}} \mathbb{W}_1^{i,j} * \mathbb{W}_2^{j,m}\right) * \curvearrowleft_i\}_{m=1}^{c_\mathbb{Z}}$ that combines the two layers. Here, k_1 and k_2 are the kernel sizes of the convolutions, and $*$ denotes the convolution operation. This new layer effectively merges the operations of g_1 and g_2 into a single convolutional layer, eliminating the non-linear activation in between.

[7]:
```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from copy import deepcopy


def fold_linear_conv_sequences(
    model,
    mean_preacts_conv,
    threshold=0.1,
):
    """
    Fold Conv-BN-ReLU-Conv sequences when the activation is near-linear.

    Args:
        model (nn.Module): ResNet-like model (unchanged architecture)
        mean_preacts_conv (dict): {conv_layer_name: mean preactivation}
        threshold (float): >= threshold => ReLU considered linear

    Returns:
        folded_model (nn.Module)
        folded_pairs (list of tuples)
```

```python
    """

    model = deepcopy(model)
    folded_pairs = []

    print("\n[Layer Folding] Starting folding pass")
    print(f"[Layer Folding] Linearity threshold: {threshold}\n")

    # ------------------------------------------------------------
    # Helper: fold BN into Conv
    # ------------------------------------------------------------
    def fold_bn_into_conv(conv, bn):
        print(f"    Folding BatchNorm into Conv ({conv.out_channels} channels)")

        W = conv.weight
        if conv.bias is None:
            bias = torch.zeros(W.size(0), device=W.device)
        else:
            bias = conv.bias

        gamma = bn.weight
        beta = bn.bias
        mean = bn.running_mean
        var = bn.running_var
        eps = bn.eps

        std = torch.sqrt(var + eps)
        W_folded = W * (gamma / std).reshape(-1, 1, 1, 1)
        b_folded = beta + (bias - mean) * gamma / std

        conv.weight.data.copy_(W_folded)
        conv.bias = nn.Parameter(b_folded)

    # ------------------------------------------------------------
    # Helper: fold Conv → Conv
    # ------------------------------------------------------------
    def fold_convs(conv1, conv2):
        print(
            f"    Folding Conv layers: "
            f"{conv1.in_channels}→{conv1.out_channels}→{conv2.out_channels}"
        )

        W1 = conv1.weight.data
        W2 = conv2.weight.data

        C_out, C_mid, k2, _ = W2.shape
        _, C_in, k1, _ = W1.shape
```

```
        W_fold = torch.zeros(
            (C_out, C_in, k1 + k2 - 1, k1 + k2 - 1),
            device=W1.device,
        )

        for m in range(C_out):
            for i in range(C_in):
                acc = 0
                for j in range(C_mid):
                    acc = acc + F.conv2d(
                        W1[j, i].unsqueeze(0).unsqueeze(0),
                        W2[m, j].unsqueeze(0).unsqueeze(0),
                    ).squeeze()
                W_fold[m, i] = acc

        new_conv = nn.Conv2d(
            in_channels=C_in,
            out_channels=C_out,
            kernel_size=W_fold.shape[-1],
            padding=W_fold.shape[-1] // 2,
            bias=False,
        )
        new_conv.weight.data.copy_(W_fold)
        return new_conv

    # ------------------------------------------------------------
    # Main traversal
    # ------------------------------------------------------------
    for module_name, block in model.named_modules():
        if not isinstance(block, nn.Sequential):
            continue

        print(f"\n[Inspecting block] {module_name}")

        # Block is a sequential with 2 basic blocks of the ResNet architecture.␣
↪Iterate over them.
        for idx, module in enumerate(block):
            # print(f" Inspecting module: {module}")

            if not hasattr(module, 'conv1') or not hasattr(module, 'bn1') or not␣
↪hasattr(module, 'relu') or not hasattr(module, 'conv2'):
                print("  Not a Conv-BN-ReLU-Conv block → skipping")
                continue

            conv1 = module.conv1
            bn1 = module.bn1
```

```
            relu = module.relu
            conv2 = module.conv2

            if not (
                isinstance(conv1, nn.Conv2d)
                and isinstance(bn1, nn.BatchNorm2d)
                and isinstance(relu, nn.ReLU)
                and isinstance(conv2, nn.Conv2d)
            ):
                print("  Not Conv-BN-ReLU-Conv → skipping")
                continue

            conv1_name = f"{module_name}.{idx}.conv1"
            mean_act = mean_preacts_conv.get(conv1_name, None)

            print(f"  Found Conv-BN-ReLU-Conv at {conv1_name}")

            if mean_act is None:
                print("    No preactivation stats → skipping")
                continue

            print(f"    Mean preactivation: {mean_act:.4f}")

            if mean_act < threshold:
                print("    Below threshold → ReLU not linear")
                continue

            # Validate foldability
            if (
                conv1.stride != (1, 1)
                or conv2.stride != (1, 1)
                or conv1.groups != 1
                or conv2.groups != 1
            ):
                print("    Stride/groups incompatible → skipping")
                continue

            print("    Linearity condition satisfied")
            print("    Removing BatchNorm and ReLU, folding Convs")

            # ----------------------------------------------------
            # Fold BN → Conv1
            # ----------------------------------------------------
            # fold_bn_into_conv(conv1, bn1)

            # ----------------------------------------------------
            # Fold Conv1 → Conv2
```

```
            # ----------------------------------------------------
            new_conv = fold_convs(conv1, conv2)

            # ----------------------------------------------------
            # Replace modules
            # ----------------------------------------------------
            setattr(module, 'conv1', new_conv)
            setattr(module, 'bn1', nn.Identity())
            setattr(module, 'relu', nn.Identity())
            setattr(module, 'conv2', nn.Identity())

            folded_pairs.append((conv1_name, f"{module_name}.{idx}.conv2"))

            print("    Folding complete")

    print(f"\n[Layer Folding] Done. Folded {len(folded_pairs)} layer pairs.\n")

    return model, folded_pairs



pruned_model, folded_layers = fold_linear_conv_sequences(model,␣
 ↪mean_preacts_conv, threshold=-0.1)
print("Folded layers:")
for layer in folded_layers:
    print(layer)
```

```
[Layer Folding] Starting folding pass
[Layer Folding] Linearity threshold: -0.1


[Inspecting block] layer1
  Found Conv-BN-ReLU-Conv at layer1.0.conv1
    Mean preactivation: -0.0341
    Linearity condition satisfied
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete
  Found Conv-BN-ReLU-Conv at layer1.1.conv1
    Mean preactivation: -0.0836
    Linearity condition satisfied
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete

[Inspecting block] layer2
  Found Conv-BN-ReLU-Conv at layer2.0.conv1
```

```
    Mean preactivation: -0.0673
    Stride/groups incompatible → skipping
  Found Conv-BN-ReLU-Conv at layer2.1.conv1
    Mean preactivation: -0.2103
    Below threshold → ReLU not linear

[Inspecting block] layer2.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer3
  Found Conv-BN-ReLU-Conv at layer3.0.conv1
    Mean preactivation: -0.1148
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer3.1.conv1
    Mean preactivation: -0.2375
    Below threshold → ReLU not linear

[Inspecting block] layer3.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer4
  Found Conv-BN-ReLU-Conv at layer4.0.conv1
    Mean preactivation: -0.2257
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer4.1.conv1
    Mean preactivation: -0.2417
    Below threshold → ReLU not linear

[Inspecting block] layer4.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Layer Folding] Done. Folded 2 layer pairs.

Folded layers:
('layer1.0.conv1', 'layer1.0.conv2')
('layer1.1.conv1', 'layer1.1.conv2')
```

[8]:
```python
# Showcase the new model architecture after folding
print(pruned_model)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1,
ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2),
bias=False)
      (bn1): Identity()
      (relu): Identity()
      (conv2): Identity()
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2),
bias=False)
      (bn1): Identity()
      (relu): Identity()
      (conv2): Identity()
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
```

```
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (layer3): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=1000, bias=True)
)
```

[9]:
```python
# Verify that the pruned model still works
pruned_outputs = verify_model_and_data_loader(pruned_model, val_loader)
print(pruned_outputs.shape)
```

```
torch.Size([128, 1000])
```

[10]:
```python
# Do small finetuneing pass to recover accuracy
def finetune_model(model, data_loader, device='cuda', epochs=1, lr=1e-4):
    model = model.to(device).train()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = torch.nn.CrossEntropyLoss()
    for epoch in range(epochs):
        print(f"Finetuning epoch {epoch+1}/{epochs}")
        for inputs, labels in tqdm(data_loader, desc="Finetuning", leave=False):
            inputs = inputs.to(device)
            labels = labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
    return model

pruned_model = finetune_model(pruned_model, train_loader, epochs=5, lr=1e-4)
```

```
Finetuning epoch 1/5
```

```
Finetuning epoch 2/5


Finetuning epoch 3/5


Finetuning epoch 4/5


Finetuning epoch 5/5
```

[11]:
```python
# Do small validation accuracy check
def validate_model(model, data_loader, device='cuda', num_batches=10):
    model = model.to(device).eval()
    correct = 0
    total = 0
    with torch.no_grad():
        progress_bar = tqdm(total=min(num_batches, len(data_loader)),␣
 ↪desc="Validating model", leave=False)
        for inputs, labels in data_loader:
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

            progress_bar.update(1)
            if total >= num_batches * data_loader.batch_size:
                break
    accuracy = correct / total
    progress_bar.close()
    return accuracy

original_acc = validate_model(model, val_loader, num_batches=100)
pruned_acc = validate_model(pruned_model, val_loader, num_batches=100)
print(f"Original model accuracy (sampled): {original_acc*100:.2f}%")
print(f"Pruned model accuracy (sampled): {pruned_acc*100:.2f}%")
```

```
Original model accuracy (sampled): 75.80%
Pruned model accuracy (sampled): 64.70%
```

1.3 Average of three experiments with fixed random seed

Now we will rerun the entire experiment three times with three fixed random seeds to get an average performance indication.

[12]:
```
def do_fixed_experiments(seeds, verbose=True, save_path=None):
    """Run the entire experiment with fixed seeds from a provided list.
    Function can report average accuracies over the seeds.
    It can also write the results to a file for later analysis.
    seeds (list of int): List of random seeds to use.
    verbose (bool): Whether to print detailed output.
    save_path (str or None): If provided, path to save results to a csv file.
    """
    original_accuracies = []
    pruned_accuracies = []

    if save_path is not None:
        with open(save_path, 'w') as f:
            f.write("Seed,Original Accuracy (%),Pruned Accuracy (%)\n")

    for seed in seeds:
        if verbose:
            print(f"\n=== Running experiment with seed {seed} ===\n")

        # Set random seed for reproducibility
        torch.manual_seed(seed)
        np.random.seed(seed)

        # Reload model and data loaders
        model = resnet18(weights=ResNet18_Weights.DEFAULT).cuda().eval()
        train_ds = make_subset(ImageFolder("./data/imagenet_train",␣
↪transform=train_tfms), fraction=0.05, seed=seed)
        train_loader = DataLoader(train_ds, batch_size=256, shuffle=True,␣
↪num_workers=0, pin_memory=True)
        val_loader = DataLoader(val_ds, batch_size=128, shuffle=False,␣
↪num_workers=0, pin_memory=True)

        # Retrieve mean preactivations
        mean_preacts = retrieve_mean_preactivations(model, val_loader)
        mean_preacts_conv = {}
        for name, module in model.named_modules():
            if isinstance(module, torch.nn.ReLU):
                mean = mean_preacts.get(name, 0.0)
                layer = name.split('relu')[0]
                mean_preacts_conv[layer + 'conv1'] = mean
```

```
        # Fold linear conv sequences
        pruned_model, folded_layers = fold_linear_conv_sequences(model,␣
↪mean_preacts_conv, threshold=-0.1)

        # Finetune pruned model
        pruned_model = finetune_model(pruned_model, train_loader, epochs=5,␣
↪lr=1e-4)

        # Validate both models
        original_acc = validate_model(model, val_loader, num_batches=100)
        pruned_acc = validate_model(pruned_model, val_loader, num_batches=100)

        original_accuracies.append(original_acc)
        pruned_accuracies.append(pruned_acc)
        if verbose:
            print(f"Seed {seed} - Original accuracy: {original_acc*100:.2f}%,␣
↪Pruned accuracy: {pruned_acc*100:.2f}%")
        if save_path is not None:
            with open(save_path, 'a') as f:
                f.write(f"{seed},{original_acc*100:.2f},{pruned_acc*100:.2f}\n")

    avg_original_acc = sum(original_accuracies) / len(original_accuracies)
    avg_pruned_acc = sum(pruned_accuracies) / len(pruned_accuracies)
    if verbose:
        print(f"\n=== Average results over seeds ===")
        print(f"Average original accuracy: {avg_original_acc*100:.2f}%")
        print(f"Average pruned accuracy: {avg_pruned_acc*100:.2f}%")
    return avg_original_acc, avg_pruned_acc

seeds = [42, 66, 762]
avg_orig_acc, avg_pruned_acc = do_fixed_experiments(seeds, verbose=True,␣
↪save_path="cnn_linearity_pruning_results.csv")
```

```
=== Running experiment with seed 42 ===




[Layer Folding] Starting folding pass
[Layer Folding] Linearity threshold: -0.1


[Inspecting block] layer1
  Found Conv-BN-ReLU-Conv at layer1.0.conv1
    Mean preactivation: -0.0341
    Linearity condition satisfied
```

```
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete
  Found Conv-BN-ReLU-Conv at layer1.1.conv1
    Mean preactivation: -0.0836
    Linearity condition satisfied
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete

[Inspecting block] layer2
  Found Conv-BN-ReLU-Conv at layer2.0.conv1
    Mean preactivation: -0.0673
    Stride/groups incompatible → skipping
  Found Conv-BN-ReLU-Conv at layer2.1.conv1
    Mean preactivation: -0.2103
    Below threshold → ReLU not linear

[Inspecting block] layer2.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer3
  Found Conv-BN-ReLU-Conv at layer3.0.conv1
    Mean preactivation: -0.1148
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer3.1.conv1
    Mean preactivation: -0.2375
    Below threshold → ReLU not linear

[Inspecting block] layer3.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer4
  Found Conv-BN-ReLU-Conv at layer4.0.conv1
    Mean preactivation: -0.2257
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer4.1.conv1
    Mean preactivation: -0.2417
    Below threshold → ReLU not linear

[Inspecting block] layer4.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Layer Folding] Done. Folded 2 layer pairs.
```

```
Finetuning epoch 1/5


Finetuning epoch 2/5


Finetuning epoch 3/5


Finetuning epoch 4/5


Finetuning epoch 5/5


Seed 42 - Original accuracy: 75.80%, Pruned accuracy: 65.06%

=== Running experiment with seed 66 ===




[Layer Folding] Starting folding pass
[Layer Folding] Linearity threshold: -0.1


[Inspecting block] layer1
  Found Conv-BN-ReLU-Conv at layer1.0.conv1
    Mean preactivation: -0.0341
    Linearity condition satisfied
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete
  Found Conv-BN-ReLU-Conv at layer1.1.conv1
    Mean preactivation: -0.0836
    Linearity condition satisfied
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete

[Inspecting block] layer2
  Found Conv-BN-ReLU-Conv at layer2.0.conv1
    Mean preactivation: -0.0673
    Stride/groups incompatible → skipping
  Found Conv-BN-ReLU-Conv at layer2.1.conv1
    Mean preactivation: -0.2103
    Below threshold → ReLU not linear
```

```
[Inspecting block] layer2.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer3
  Found Conv-BN-ReLU-Conv at layer3.0.conv1
    Mean preactivation: -0.1148
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer3.1.conv1
    Mean preactivation: -0.2375
    Below threshold → ReLU not linear

[Inspecting block] layer3.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer4
  Found Conv-BN-ReLU-Conv at layer4.0.conv1
    Mean preactivation: -0.2257
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer4.1.conv1
    Mean preactivation: -0.2417
    Below threshold → ReLU not linear

[Inspecting block] layer4.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Layer Folding] Done. Folded 2 layer pairs.

Finetuning epoch 1/5


Finetuning epoch 2/5


Finetuning epoch 3/5


Finetuning epoch 4/5


Finetuning epoch 5/5


Seed 66 - Original accuracy: 75.80%, Pruned accuracy: 65.01%
```

```
=== Running experiment with seed 762 ===




[Layer Folding] Starting folding pass
[Layer Folding] Linearity threshold: -0.1


[Inspecting block] layer1
  Found Conv-BN-ReLU-Conv at layer1.0.conv1
    Mean preactivation: -0.0341
    Linearity condition satisfied
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete
  Found Conv-BN-ReLU-Conv at layer1.1.conv1
    Mean preactivation: -0.0836
    Linearity condition satisfied
    Removing BatchNorm and ReLU, folding Convs
    Folding Conv layers: 64→64→64
    Folding complete

[Inspecting block] layer2
  Found Conv-BN-ReLU-Conv at layer2.0.conv1
    Mean preactivation: -0.0673
    Stride/groups incompatible → skipping
  Found Conv-BN-ReLU-Conv at layer2.1.conv1
    Mean preactivation: -0.2103
    Below threshold → ReLU not linear

[Inspecting block] layer2.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer3
  Found Conv-BN-ReLU-Conv at layer3.0.conv1
    Mean preactivation: -0.1148
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer3.1.conv1
    Mean preactivation: -0.2375
    Below threshold → ReLU not linear

[Inspecting block] layer3.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Inspecting block] layer4
```

```
  Found Conv-BN-ReLU-Conv at layer4.0.conv1
    Mean preactivation: -0.2257
    Below threshold → ReLU not linear
  Found Conv-BN-ReLU-Conv at layer4.1.conv1
    Mean preactivation: -0.2417
    Below threshold → ReLU not linear

[Inspecting block] layer4.0.downsample
  Not a Conv-BN-ReLU-Conv block → skipping
  Not a Conv-BN-ReLU-Conv block → skipping

[Layer Folding] Done. Folded 2 layer pairs.

Finetuning epoch 1/5


Finetuning epoch 2/5


Finetuning epoch 3/5


Finetuning epoch 4/5


Finetuning epoch 5/5


Seed 762 - Original accuracy: 75.80%, Pruned accuracy: 64.82%

=== Average results over seeds ===
Average original accuracy: 75.80%
Average pruned accuracy: 64.96%
```